

Da Implementação de Julho ao Portfólio Atual

O que mudou em cada uma das 18 políticas de reposição do AIPE

Matheus Inácio Batista Santos

Apresentação técnica de acompanhamento

Universidade Federal de Alagoas (UFAL)

Instituto de Computação (IC)

Programa de Pós-Graduação em Informática (PPGI)

Orientadores: Dr. Rian G. S. Pinheiro e Dr. Bruno C. S. Nogueira

1. Por que reimplementar?
2. O simulador: de laço Python a lote vetorizado
3. Políticas clássicas: o que ficou e o que entrou
4. Meta-heurísticas: GA, SA, PSO, DE
5. A função de aptidão: de soma ponderada a restrição com risco
6. Aprendizado por reforço: DQN, PPO, SARSA
7. Híbridas GA-RL
8. Infraestrutura e hiperparâmetros
9. Portfólio final e próximos passos

Por que reimplementar?

Por que reimplementar o portfólio inteiro?

Não foi troca de biblioteca. Foram dois defeitos que afetam resultados já reportados.

- A política rotulada **PPO não otimizava o objetivo do PPO**: a atualização do ator estava matematicamente errada (próximo slide).
- A **aptidão das meta-heurísticas** (GA/SA/PSO/DE) usava soma ponderada com pesos fixos – ponto de operação sensível à escala do custo, que varia por ordens de grandeza entre séries.

Consequência prática: qualquer correção pontual exigiria retreinar RL e recalibrar meta-heurísticas de qualquer forma. A decisão foi aproveitar a reescrita para também:

- vetorizar o simulador (laço Python → lote em PyTorch);
- ampliar o portfólio de 12 para **18 políticas**, cobrindo estado da arte publicado onde ele existe;

Os dois defeitos que motivaram a reescrita

	Antes (jul/2026)	Depois (atual)
PPO	Ator atualizado somando a vantagem A_t diretamente aos logits-alvo e treinado por erro quadrático . Isso não é o gradiente da <i>surrogate function</i> recortada do PPO.	Gradiente correto por autograd sobre a razão de verossimilhanças $r_t = \pi_{\text{novo}}/\pi_{\text{antigo}}$, com clipping explícito e GAE (λ).
Aptidão GA/SA/PSO/DE	Soma ponderada fixa $\lambda_1 \text{ NS} - \lambda_2 \text{ CTI}$. O ponto de operação depende da escala do CTI , que varia até 124× entre a maior e a menor loja do recorte piloto.	Formulação restrita (Eq. 4.2 da proposta): $\min \text{CTI}$ s.a. $\text{NS} \geq \alpha_{\min}$, com penalidade relativa ao próprio CTI – adimensional, comparável entre séries.

Por que isso importa para os resultados já reportados

A degeneração observada no Experimento 1 (frequência de pedidos $\text{FP} = 0,98$ para a política PPO – pedir quase todo ciclo) é **consistente com o defeito do gradiente**, não com uma limitação real do método PPO.

Panorama: o que mudou em cada família

Família	O que mudou	Natureza
Clássicas (EOQ, (s, S), Jornaleiro)	Nada – mantidas como baseline	inalterada
SOTA clássicas (PIL, CappedBaseStock, BDN)	Não existiam	nova
Zabraoui (Min-Max, Fixed Interval, Vendor-Resp.)	Não existiam	nova
GA	Vetorizado; +elitismo; aptidão corrigida	corrigida + vetorizada
SA	<code>scipy.dual_annealing</code> → Metropolis explícito, cadeias paralelas	reescrita
PSO	Sem constrição → fator de constrição de Clerc & Kennedy	corrigida
DE	<code>scipy.differential_evolution</code> → DE/rand/1/bin vetorizado	reescrita
DQN	Rede NumPy manual → Double DQN dueling (PyTorch)	reescrita
PPO	Gradiente do ator corrigido + GAE	corrigida
SARSA	Tabular 1D → Expected SARSA, discretização 2D	reescrita
GA-DQN / GA-PPO	Piso de quantidade (<code>floor_ratio</code>) adicionado	ajustada

Vermelho = corrige defeito. Verde = nova política. Azul-petróleo = reescrita sem mudança de defeito

conhecido (vetorização/robustez).

O simulador

O simulador: de laço Python a lote vetorizado

Antes (jul/2026)

InventoryEnv: uma trajetória por chamada.

- Um indivíduo/partícula = 1 simulação de T ciclos, em laço Python.
- População 100×50 gerações do GA = **5.000 simulações sequenciais**.
- Cada 'step' é uma chamada de função Python por ciclo.

Depois (atual)

BatchInventoryEnv: B trajetórias em paralelo.

- Toda a população avança **um ciclo por vez, em bloco** (operações tensoriais de largura B).
- Mesma configuração do GA = **50 chamadas** de simulação de largura 100, não 5.000 chamadas de largura 1.
- Custo passa de $O(B \times T)$ chamadas Python para $O(T)$ operações tensoriais.

O que NÃO mudou

Ordem das operações por ciclo (chegada $\rightarrow$ demanda $\rightarrow$ custo), *lost sales* (demanda não atendida é perdida, não acumulada), pipeline de $L+1$ posições, e os cinco KPIs (CTI, NS,

Garantia: o novo simulador reproduz o antigo

Reescrever o simulador é o maior risco desta mudança – por isso ele foi validado numericamente antes de qualquer política ser reimplementada sobre ele.

- Teste de equivalência (`tests/test_env_equivalence.py`): mesma política, mesma demanda, mesma semente, comparando `InventoryEnv` escalar contra `BatchInventoryEnv` em **30 casos**.
- Erro relativo máximo: $3,7 \times 10^{-7}$ em CTI, NS, TR, BE e FP – compatível com precisão de ponto flutuante, não com diferença de lógica.

Consequência metodológica

Qualquer diferença nos resultados entre a versão de julho e a atual é atribuível às políticas e à função de aptidão – **não a uma mudança silenciosa nas regras do ambiente de simulação.**

Políticas clássicas: o que ficou e o que entrou

As três clássicas originais: nada mudou

Política	Regra	Parâmetros
EOQ	$Q^* = \sqrt{2DK/h}$; pede quando $I + \text{pipeline} \leq \text{ROP}$	$\text{ROP} = \mu L + z\sigma\sqrt{L}$
(s, S)	Banda min-max: pede até S quando posição $\leq s$	$s = \mu L + z\sigma\sqrt{L}$; $S = s + Q_{\text{EOQ}}$
Jornaleiro	$Q^* = \hat{\mu} + z\hat{\sigma}$ (quantil crítico fixo)	$z = 1,28$ (SL alvo 90%)

Por que preservar

São o **baseline do varejo** e o denominador de todas as comparações já reportadas no Capítulo 5. Mudar sua fórmula invalidaria a leitura de qualquer resultado anterior que as usa como referência.

Novo (ago/2026): três políticas clássicas em versão estado da arte

Complementam (não substituem) EOQ/(s, S)/Jornaleiro – mesmo slot conceitual, referência publicada mais recente.

Política	Ideia central	Referência
PIL (<i>Projected Inventory Level</i>)	$Q_t = \max(0, S - \text{PIL}_t)$, projetando estoque L ciclos à frente por reamostragem empírica, com $\max(0, \cdot)$ em cada ciclo intermediário (viés para baixo sob <i>lost sales</i> se projetado linearmente)	van Jaarsveld & Arts, <i>Oper. Res.</i> 72(5), 2024
Capped base-stock	$Q_t = \min(\text{cap}, \max(0, S - \text{IP}_t))$ – híbrido entre base-stock e pedido constante; piso honesto para comparar RL	Xin, <i>Oper. Res.</i> 69(1), 2021
Big Data Newsvendor	Regressão quantílica no quantil crítico $\tau = c_s/(c_s+h)$ sobre 8 características causais de intermitência (não assume distri-	Ban & Rudin, <i>Oper. Res.</i> 67(1), 2019

Novo (ago/2026): três heurísticas do artigo-base (Zabraoui et al., 2025)

Seção 3.5 do artigo-base define quatro políticas de referência; apenas (s, S) já existia no nosso portfólio. As outras três entram aqui.

Política	Regra adotada
Min-Max	Pede até M quando posição $\leq m$; $m = \mu L$ (lead time L), $M = m + \mu \cdot \text{cobertura}$
Fixed Interval Replenishment	Revisão a cada R ciclos; repõe até $S = \mu(R+L) + z\sigma\sqrt{R+L}$
Vendor-Responsive	$s_t = \mu L + z_t \sigma \sqrt{L}$, com $z_t = z_0(1 + \kappa(\alpha_{\text{alvo}} - SL_t))$ – realimentação pelo serviço realizado até o ciclo t

Ressalva de fidelidade, registrada no código

Min-Max e Fixed Interval são descritas no artigo em **uma frase cada, sem fórmula**.
Vendor-Responsive é a mais subespecificada (“*adapts order timing based on supplier lead times and historical service levels*”). As fórmulas acima são leitura razoável dessas descrições, **não transcrição**

10/26

Meta-heurísticas

Algoritmo Genético – Antes (jul/2026)

Biblioteca DEAP. Representação real $\theta = (\text{ROP}, Q, \text{SS})$.

Operador	Implementação
População	100 indivíduos, inicializados uniformemente nos limites de busca
Gerações	50
Seleção	Torneio, tamanho 3 (<code>tools.selTournament</code>)
Cruzamento	<i>Blend crossover</i> $\alpha = 0,5$ (<code>tools.cxBlend</code>), prob. 0,8
Mutação	Gaussiana, $\sigma =$ maior amplitude do espaço de busca /10, prob. 0,05 por gene (<code>tools.mutGaussian</code>)
Elitismo	Nenhum – <code>HallOfFame(1)</code> só registra o melhor, não o protege do crossover
Avaliação	1 simulação completa de T ciclos por indivíduo, em laço Python
Aptidão	Soma ponderada $\lambda_1 \text{NS} - \lambda_2 \text{CTI}$, pesos fixos

Algoritmo Genético – Depois (atual)

PyTorch (TorchGA), sem DEAP. Mesma representação $\theta = (\text{ROP}, Q, \text{SS})$.

Operador	Implementação
População	100 (inalterado)
Gerações	50 na Bahia/"bot"; 10 no M5 (orçamento de tempo, slide 8)
Seleção	Torneio, tamanho 3 (inalterado), mas vetorizada: <code>torch.randint</code> sorteia 100×3 candidatos de uma vez, <code>argmin</code> por linha decide os vencedores
Cruzamento	<i>Blend crossover</i> $\alpha \sim U(0,1)$ por par e por gene (inalterado em essência), aplicado a todos os 50 pares simultaneamente com <code>torch.where</code>
Mutação	Gaussiana, mesmo desvio-padrão relativo ao espaço de busca, prob. 0,05 por gene (inalterado), amostrada para a população inteira de uma vez
Elitismo	Novo: o melhor indivíduo histórico substitui o pior da nova geração

Simulated Annealing – Antes (jul/2026) vs. Depois (atual)

Antes (jul/2026)

`scipy.optimize.dual_annealing`

- Cronograma de temperatura e critério de aceitação **internos à biblioteca** – caixa-preta com parâmetros próprios, não os documentados na proposta.
- Refinamento local por Nelder-Mead a cada iteração.
- Uma cadeia, avaliação sequencial.

Depois (atual)

TorchSA: cronograma explícito

- $T_0 = 1000$, $\alpha_{\text{resfr.}} = 0,95$ (Seção 4.4 da proposta), $T_{k+1} = \alpha T_k$.
- Critério de Metropolis auditável: aceita se $\Delta C < 0$, ou com probabilidade $e^{-\Delta C / T_k}$.
- **32 cadeias independentes** (`n_chains=32`) em paralelo, mesmo cronograma, saída = melhor entre elas.

Por que isso é uma reescrita, não só uma correção

SA é inerentemente sequencial. Vetorizar sem descaracterizar o método significa rodar várias cadeias **independentes** com o mesmo cronograma, em vez de fingir paralelismo dentro de uma cadeia. O ganho de robustez vem de reportar o melhor entre 32 buscas, não de uma cadeia mais rápida.

Particle Swarm Optimization – Antes (jul/2026) vs. Depois (atual)

Antes (jul/2026)

NumPy puro, laço Python por partícula

- 40 partículas $\times$ 80 iterações.
- $w = 0,7$, $c_1 = c_2 = 1,5$, **sem** fator de constrição.
- Velocidade sem limite explícito – pode divergir.
- Avaliação: laço for i in `range(n_particles)`, uma simulação por partícula.

Depois (atual)

TorchPSO, avaliação em lote

- 40 $\times$ 80 (**inalterado**).
- Mesmos w , c_1 , c_2 + **fator de constrição de Clerc & Kennedy (2002)**:
$$\chi = \frac{2}{|2 - \phi - \sqrt{\phi^2 - 4\phi}|}, \phi = c_1 + c_2.$$
- $v \leftarrow \chi [wv + c_1 r_1 (p_{\text{best}} - x) + c_2 r_2 (g_{\text{best}} - x)]$
– converge sem *clamp* arbitrário de velocidade.

Sintoma observado que motivou a correção

Nos resultados anteriores, Q frequentemente “colava” no limite superior do espaço de busca – assinatura clássica de velocidade divergente em PSO sem constrição.

Differential Evolution – Antes (jul/2026) vs. Depois (atual)

Antes (jul/2026)

`scipy.optimize.differential_evolution`

- `popsize=15` (multiplicador interno do scipy → população efetiva maior, não documentada explicitamente).
- Estratégia padrão do scipy (`best1bin`), mutação em faixa (0,5, 1,0).
- `polish=True`: refinamento local por L-BFGS-B ao final – passo extra fora da formulação DE canônica.

Depois (atual)

TorchDE: DE/rand/1/bin explícito (Storn & Price, 1997)

- População = $15 \times 3 = 45$ (**tamanho equivalente**, agora explícito no código, sem multiplicador implícito).
- Mutação: $v_i = x_{r_1} + F(x_{r_2} - x_{r_3})$, $F = 0,8$.
- Cruzamento binomial com $CR = 0,9$, garantindo ≥ 1 dimensão herdada do mutante (`jrand`).
- Seleção **gulosa** elemento a elemento – **sem** polimento local: fiel à formulação DE/rand/1/bin canônica.

Toda a geração avaliada em uma chamada; a seleção gulosa é um único `torch.where` – não

A função de aptidão

A função de aptidão tem sua própria evolução

Três versões, em ordem cronológica – não é uma correção pontual, é uma linha evolutiva.

1. **Antes (jul/2026)** – soma ponderada (Eq. 3 do artigo-base):
 $\text{Fitness} = \lambda_1 \text{NS} - \lambda_2 \text{CTI}$, $(\lambda_1, \lambda_2) = (1, 0; 0, 0001)$ fixos.
2. **Intermediário (ago/2026, reimplementação)** – formulação restrita (Eq. 4.2 da proposta): $\min \text{CTI}$ s.a. $\text{NS} \geq \alpha_{\min}$, penalidade relativa ao próprio CTI.
3. **Novo (ago/2026) (esta sessão)** – qualquer um dos dois modos + **termos de risco** baseados em TR e BE (próximos slides).

Por que a versão 1 é problemática

O CTI varia por **ordens de grandeza** entre séries (razão de $124\times$ entre a maior e a menor loja no estudo piloto). Pesos fixos fazem a MESMA configuração privilegiar serviço em séries de baixo volume e custo em séries de alto volume – contaminando a comparação entre políticas.

Estágio 2: formulação restrita (Eq. 4.2)

Objetivo, a minimizar:

$$\text{custo} = \text{CTI} + \delta \cdot p \cdot \max(\text{CTI}, 1)$$

$$\delta = \max(0, \alpha_{\min} - \text{NS})$$

em que $\alpha_{\min} = 0,70$ e $p = 10$ (peso de penalidade).

Por que isso resolve o problema de escala

- A penalidade é **proporcional ao próprio CTI** (via $\max(\text{CTI}, 1)$), não uma constante fixa.
- Candidatos viáveis ($\text{NS} \geq \alpha_{\min}$) são ordenados por $-\text{CTI}$ puro – sem distorção de escala.
- Mesma fórmula usada nas versões escalar (`policies._eval_static`) e vetorizada (`constrained_cost`) – **idênticas por construção**.

Retrocompatibilidade: `fitness_mode="weighted"` preserva o comportamento antigo, para reproduzir os resultados já reportados no Capítulo 5 se necessário.

Novo (ago/2026): termos de risco (TR, BE) somados à aptidão

CTI só registra o custo de ruptura **JÁ REALIZADO** na trajetória simulada. TR e BE capturam fragilidade que o CTI sozinho não pune.

$$\text{fitness} += \underbrace{\lambda_{\text{TR}} \cdot \text{TR} \cdot (1 - \text{NS}) \cdot \text{CTI}_{\text{ref}}}_{\text{risco de ruptura}} + \underbrace{\lambda_{\text{BE}} \cdot \max(\text{BE} - 1, 0) \cdot \text{CTI}_{\text{ref}}}_{\text{compra excessiva}}$$

TR $\times$ **(1-NS)**

- TR = fração de ciclos em que **falta produto pro cliente** (probabilidade de ruptura).
- Modulado por (1-NS): risco composto cresce quando serviço **já ruim** E ruptura **frequente** coincidem.
- BE = $\text{Var}(Q)/\text{Var}(D)$.
- BE ≤ 1 : pedido tão ou mais suave que a demanda – **não punido**.
- BE > 1 : amplificação – só o **excesso** acima de 1 é punido (“compra excessiva”).

$\lambda_{\text{TR}} = \lambda_{\text{BE}} = 1,0$ por padrão; $\text{CTI}_{\text{ref}} = \max(\text{CTI}, 1)$, mesmo truque de escala da penalidade de NS. Soma-se a QUALQUER `fitness_mode` (constrained ou zabraoui).

Aprendizado por reforço

DQN – Antes (jul/2026) vs. Depois (atual)

Antes (jul/2026)

Rede _NN: NumPy escrito à mão

- MLP simples, *forward/backward* manuais, ReLU, sem otimizador adaptativo (SGD puro).
- DQN padrão: mesma rede escolhe e avalia a ação do próximo estado – **superestimação** de valor conhecida na literatura.
- Perda: erro quadrático médio.
- Sem *clipping* de gradiente.

Depois (atual)

DoubleDQNAgent (PyTorch)

- Arquitetura **dueling** (Wang et al., 2016):
 $Q(s, a) = V(s) + [A(s, a) - \bar{A}(s, \cdot)]$.
- **Double DQN** (van Hasselt et al., 2016):
rede *online* escolhe a ação, rede *alvo* avalia
 $- y = r + \gamma Q_{\text{alvo}}(s', \arg \max_a Q_{\text{online}}(s', a))$.
- Perda de **Huber** (`smooth_l1_loss`), Adam, *grad clipping* em 10.

Por que Double DQN dueling especificamente

Em horizontes curtos ($T = 38$ ciclos), a superestimação do DQN clássico é especialmente danosa: o agente aprende a pedir demais porque valoriza excessivamente estados de estoque

PPO – o defeito, em detalhe matemático

Objetivo correto do PPO (Schulman et al., 2017):

$$L(\theta) = -\mathbb{E} \left[\min \left(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t \right) \right], \quad r_t = \frac{\pi_{\text{novo}}(a_t | s_t)}{\pi_{\text{antigo}}(a_t | s_t)}$$

Antes (jul/2026) (o defeito)

- `tgt_a[i, A[i]] += adv[i] * min(rat[i], cl[i])`
- Soma a vantagem **diretamente aos logits-alvo** e treina o ator por **erro quadrático** contra esse alvo.
- Isso **NÃO é o gradiente** de $L(\theta)$ acima – é uma heurística sem garantia de melhoria monotônica da política.

Depois (atual) (a correção)

- Gradiente obtido por **autograd** diretamente sobre $r_t A_t$ e o termo recortado, como na formulação original.
- `policy_loss = -torch.min(s1, s2).mean()`
- **GAE**($\lambda=0,95$) para o estimador de vantagem, normalização de vantagem, recorte também da função valor, bônus de entropia.

SARSA – Antes (jul/2026) vs. Depois (atual)

Antes (jul/2026)

SARSA tabular clássico

- Tabela Q de 5×10 (5 estados de estoque $\times$ 10 ações).
- Discretização **1D**: só nível de estoque normalizado.
- Alvo TD amostra a próxima ação a' realmente tomada (ϵ -greedy) – variância de amostragem.
- $\alpha = 0,1$, $\gamma = 0,99$, $\epsilon = 0,1$.

Depois (atual)

ExpectedSARSAAgent

- Tabela Q de $(5 \times 3) \times 10$ – discretização **2D**: estoque $\times$ pedidos em trânsito (`n_pipeline_states=3`).
- Alvo = **esperança** sob a política ϵ -greedy:
 $y = r + \gamma \sum_a \pi(a|s') Q(s', a)$, não uma amostra.
- Mesmos α , γ , ϵ (**inalterados**).

Sem estado da arte publicado para este slot

Diferente de DQN e PPO, não existe estado da arte publicado em veículo relevante para SARSA aplicado a inventário. A escolha foi Expected SARSA (van Seijen et al., ADPRL 2009) – reduz a variância do SARSA amostrado mantendo o carácter on-policy, permanecendo

Híbridas GA-RL

GA-DQN / GA-PPO – Antes (jul/2026) vs. Depois (atual)

GA decide **QUANDO** pedir (limiar ROP+SS); o agente de RL decide **QUANTO** – inalterado nas duas versões.

Antes (jul/2026)

- `qty = max(rl.act(s)[1], self.Q * 0.5)`
- Piso de **50% do Q do GA** fixo, sem parametrização.
- RL de baixo nível ainda sofria do defeito do PPO (Seção 6) quando o híbrido usava GA-PPO.

Depois (atual)

- Mesmo piso, agora **parametrizado**: `floor_ratio` (padrão 0,5), configurável por `simulation.hybrid`.
- Herda **automaticamente** as correções de DQN e PPO (Seção 6) – a classe `HybridGARL` recebe qualquer agente já treinado como componente.
- Herda também o GA vetorizado com elitismo (Seção 4) na fase (i) de inicialização.

Infraestrutura e hiperparâmetros

Sobre GPU: implementada, mas raramente usada

A vetorização é o que traz o ganho de desempenho – não a GPU. E isso foi medido, não assumido.

- `core/device.py`: roteamento CPU/CUDA/MPS **consciente do tamanho do lote** – decide o acelerador só acima de um limiar medido, não por preferência fixa.
- Medição: MPS (Apple GPU) só compensa acima de **~150 mil trajetórias simultâneas**.
- Com população de 100 indivíduos (GA) ou 40 partículas (PSO), a **CPU é 35× mais rápida** – overhead de transferência CPU↔GPU domina em lotes pequenos.

Decisão de engenharia

Rotas de RL (`rl_torch.py`) fixam CPU por padrão (redes de $\sim 10^4$ parâmetros, lotes de 64 – overhead de sincronização de GPU não compensa). Meta-heurísticas

(`metaheuristics_torch.py`) consultam o limiar medido a cada execução, por tamanho de

Hiperparâmetros: alinhados a uma referência auditável

Onde **não existe** estado da arte publicado para meta-heurísticas em inventário, os hiperparâmetros foram alinhados às Tabelas 4 e 5 de Zabraoui et al. (2025) – referência única e citável, em vez de valores ajustados ad-hoc.

Parâmetro	Antes	Depois
GA crossover_prob	0,9	0,8 (Zabraoui Tab.5, faixa 0,6–0,9)
GA elitism	ausente	true (Zabraoui Sec.3.4)
DQN epsilon inicial documentado	–	Tab.4: $\epsilon = 0,2$; Sec.3.8: ≥ 1000 episódios
PPO entropy_coef	0,01	0,005 (Zabraoui Tab.4)

Isto não é “upgrade para estado da arte”

Registrado explicitamente no código (docs/references/estado_da_arte_politicas.md, Seção 3.2): **não há** estado da arte publicado em veículo relevante para meta-heurísticas aplicadas a inventário – a frente teórica migrou para aprendizado por reforço profundo.

Adotar o artigo base é a alternativa mais defensável a inventar hiperparâmetros

Portfólio final

Portfólio final: de 12 para 18 políticas

Clássicas	SOTA / Zabraoui Novo (ago/2026)	Busca & RL	Híbridas
EOQ	PIL	GA	GA-DQN
(s, S)	CappedBaseStock	SA	GA-PPO
Jornaleiro	BigDataNewsvendor	PSO	
	Min-Max	DE	
	Fixed Interval	DQN	
	Vendor-Responsive	PPO	
		SARSA	
3	6 novas	7	2

Onde isso está sendo validado agora

As 18 políticas, com a função de aptidão em seu estágio mais recente (Eq. 4.2/Zabraoui + TR/BE), estão sendo reexecutadas em **três bases**, com o mesmo critério de otimização em todas:

Base	Escopo	Status
Bahia (Experimento 2, oficial)	145 séries	concluída
“bot” (interna completa, 27 estados)	4.869 séries	em andamento
M5 (Walmart, benchmark externo, sem cortes)	30.490 séries	em andamento

Por que reexecutar a Bahia

A extensão TR/BE (estágio 3 da aptidão) foi aplicada **durante** a execução anterior da Bahia – deixando etapas já concluídas (clássica, meta-heurística, híbrida) com a aptidão antiga. Reexecutada do zero para que as **três bases otimizem pelo mesmo critério**, condição necessária para qualquer comparação entre elas.